Infraestrutura de software
Guia de Relatorios

Atenção

- **Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.**
- **O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.**
- **Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.**

1. Objetivo (3–5 linhas)

O que o enunciado pede e o que você entrega. Não explique a implementação aqui.

O enunciado pede a implementação de 4 funções (uma de inicialização e três de operação), para o gerenciamento e sincronização de múltiplas threads.

2. Checklist de Requisitos

Um requisito por linha. Sem parágrafo, se precisar explicar algo, use a coluna "como testar".

Requisito do enunciado	Status	Como testar / evidência
Inicialização correta e segura, e com Makefile	ok	make run
wait_for_car	ok	make run
load_car	ok	make run
station_on_board	ok	make run
Returns antecipados e não usar semáforos/ primitivas/busy-wait.	ok	make run

3. Como Reproduzir

Comandos reais, copiáveis. Se não rodar com o que está aqui, não é avaliado.

Compilar:

```
make clean
make run
```

Executar (exemplo):

```
./sync_car
```

4. Arquitetura (resumida)

Uma linha por arquivo, só o que é específico do seu projeto.

Arquivos e responsabilidades:

1. **sync_car.c - Implementação das funções apresentadas em sync_car.h, sendo responsável por travamento, liberação, avaliação e controle de threads que recebem e são liberadas para o uso de mutex.**
2. **sync_car.h - Implementação das struct station e do protótipos das funções desenvolvidas em sync_car.c, sendo os protótipos por detrás delas. Responsável por pela determinação base e por serem chamadas em sync_car-runner.c**

Decisões técnicas:

1- Modularizar as funções em dois arquivos, com os protótipos em sync_car.h e suas definições em sync_car.c -> Maior modularidade e menor acoplamento entre as funções.

2- Usar signal em station_on_board-> Permitir o uso do princípio de menor privilégio, apenas acordando a variável cond_t necessária (no caso, all_seats), para ser utilizada no momento adequado.

5. Estratégias e Diário de Desenvolvimento

Isto é o núcleo do relatório.

5.1 Estratégias da semana

Uma linha por abordagem diferente que você tentou. Se você não mudou de abordagem, preencha só S1.

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
1	Foco no Vscode	A estratégia principal, Cada função foi sendo desenvolvida por tentativa e erro, assim como as configurações do makefile e outras configurações primordiais.	

5.2 Diário de Tentativas

Uma linha por tentativa relevante, marcando a qual estratégia (S1/S2/S3) ela pertence, obviamente precisa ser em ordem cronológica. A coluna "Quando" precisa bater com a ordem cronológica do evidencias.log, é isso que comprova que o fluxo é real. "Hipótese/Causa" só é preenchida quando o resultado foi falha ou parcial.

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	Configurações básicas e de Makefile, a qual ainda constava com múltiplos erros	Após criar o repositório, foi iniciada a preparação dos arquivos originais e a escrita da primeira versão do makefile, a qual falhava devido a reconfiguração do makefile, como nomes equivocados e mal-uso de flags para compilação	A execução do makefile frequentemente contava com erros relacionados e causados pela configuração inadequada, mas posteriormente resolvidos.	Falhas de configuração, como sync_car não funcionando como executável, e falhas no uso das bibliotecas para executar sync_car-runner.c.	29/10	Linha 1 até linha 210 de evidências.log
1	Construção das primeiras versões do código até ser alcançada a primeira versão bem-sucedida	Após preparar o makefile e os arquivos base, foi o momento de começar a detalhar cada etapa.	Os defeitos que aconteceram foram relacionados às tentativas de refatoração e/ou falhas em nomenclatura das variáveis (como no caso específico, em que houve a troca entre a	A principal razão desencadeando as falhas, baseou-se na montagem de código (por ainda estar incompleto e	29/10-30/10	Linha 211 até Linha 587 de evidencias.log

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
		Logo, decidir utilizar 2 variaveis do tipo cond_t e 3 contadores simples, de forma a representar cada etapa: free_p (threads_recém chegadas	variável car_left e all_seats resultando em falha de operação).	instável), além de outras falhas, como uma breve confusão de responsabilidades de cada variável (neste estágio, a razão se deveu à má escolha de nomes, da minha parte, motivando também a etapa seguinte, de revisão de código).		
1	Primeira etapa de revisões de código	Após a primeira compilação bem-sucedida, foram realizadas melhorias, especialmente quanto a nome de variáveis, organização de código e a nível de montagem lógica do código, pois apesar de funcionar, ainda era bastante pouco organizado, passível de otimização e mais testes	Algumas mudanças foram feitas que causaram regressões e novas falhas (como tentar substituir broadcast por signal em wait_for_car e em load_car), além da alteração do nome da variável count para on_board e car_left para car_here).	As falhas que ocorreram se originaram em falhas de atualização e em excertos de código, como no caso de count mudada para on_board, e no reposicionamento do wait presente em load_car.	30/10-31/10	Linha 588 até 918 de evidencias.log, além do primeiro print (pré alteração de nomes das variáveis, mostrando o último caso de erro registrado).
1	Escrita do readme.md e	Além de escrever	Nesta etapa, além da elaboração do	Ocorreram erros, dentre	31/10	Da linha 919 até a linha

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
	melhorias finais do código original	o readme (no final do processo), senti ser necessário, a realização de refatoraçõ es de código,devido a falhas de falta de verificaçã o explícita (como em load_car, que apesar do else do if cobrir, ainda não constava como early-retur n)	readme, foram realizados testes com números negativos e inválidos, como quantidades menores que a passada, e verificações foram inseridas de forma a blindar o código, como a inserção de early return também em station_on_board.	eles o erro respectivo à números negativos, além de uma otimização (no caso inserir um early return em load_car), que aparece no 1º print, que não deveria ter sido posto ao final, mas facilmente corrigido, assim como um caso específico de erro de digitação de uma das variáveis (antes ainda do erro que aparece no 1º print).		1709 de evidências.lo g, além dos do segundo print, tirado no dia 31/10, o teste final pré-entrega, da versão definitiva)

Regra prática: se algo deu muito errado (bug que te travou por horas), a linha da tabela resume em 1 frase e aponta para o trecho do evidencias.log com o detalhe.

6. Evidências

A regra continua: toda afirmação relevante precisa de prova. O que muda é como capturar essa prova, para não depender de print manual toda hora.

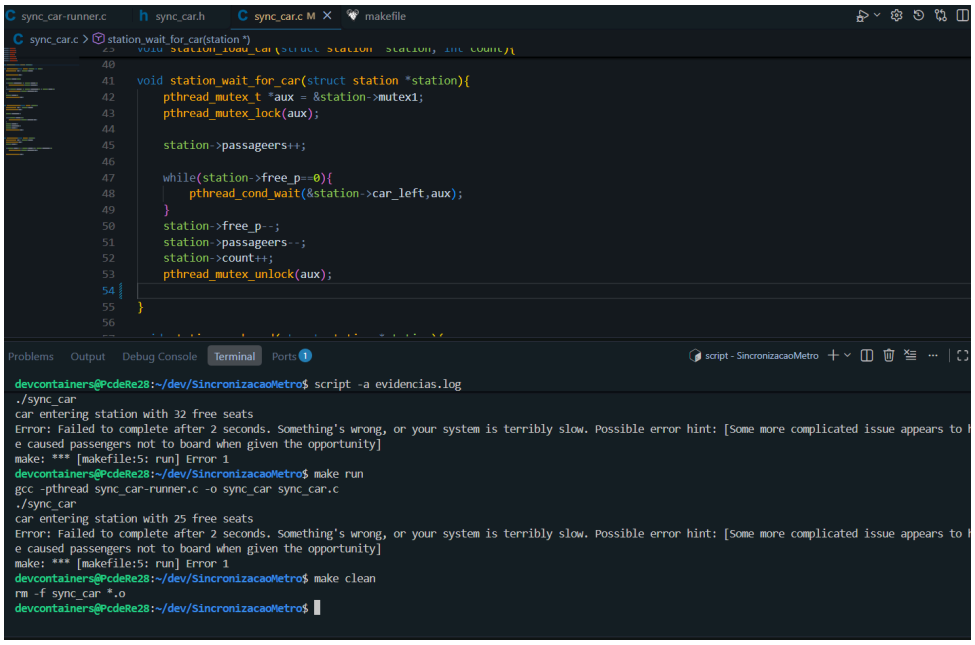
6.1 Log automático (faz o trabalho pesado por você)

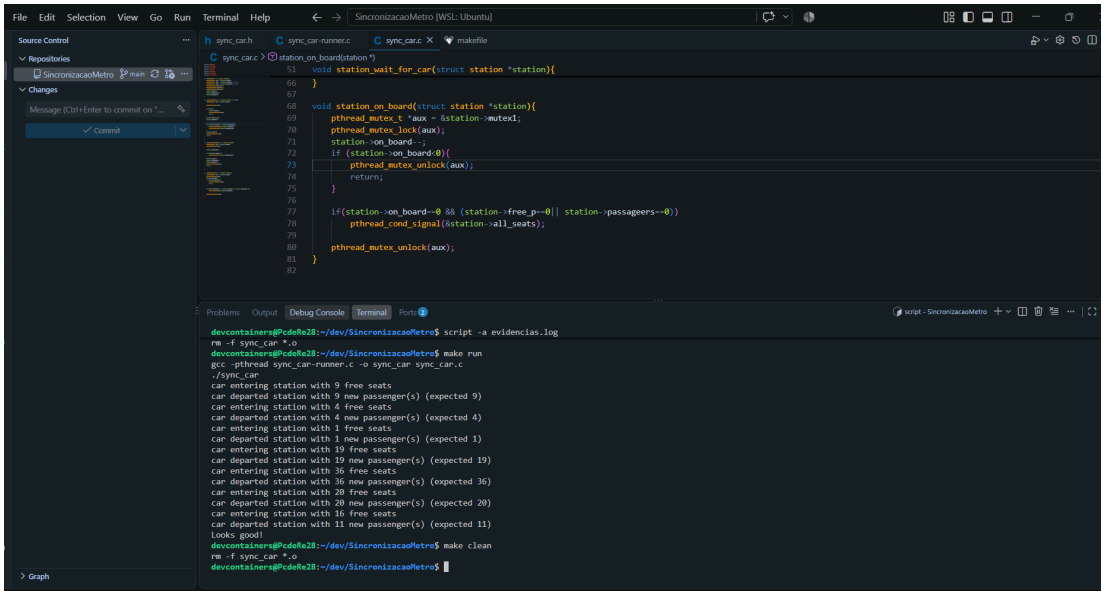
No início da sessão de trabalho, rode:

```
script -a evidencias.log
```

Isso grava automaticamente todos os comandos e saídas do terminal num arquivo, sem você precisar tirar print de cada linha. Comece a sessão com date, whoami e pwd, assim o log já tem timestamp, usuário e diretório sem esforço extra. **O evidencias.log deve ser enviado dentro do .tar**

6.2 Prints





A primeira imagem ilustra a última verificação mal-sucedida(no dia 30) pré entrega (no caso, causada por conta (apesar de não aparecer na imagem) do bloco if de early return da função load_car (que havia sido posto na posição inadequada, antes do loop de broadcast com while) que causava a não finalização correta do loop (conforme exposto na imagem, em duas tentativas). Já a segunda imagem mostra a versão final, passando no teste de verificação definitivo, neste caso já sem ocorrer falhas.

- Erro principal: o momento do bug mais relevante da semana.
- Validação final: teste passando / saída correta.

Prints extras só se algo excepcional aconteceu e o log sozinho não explica, você precisa ter esse feeling, mas posso dar uma ajuda (ex.: comportamento visual, crash com stack trace longo).

7. Uso de IA
Obrigatório preencher mesmo se a resposta for "não utilizei".

Onde usei IA (2–3 linhas): Não utilizei IA

Prompts principais (liste, sem história): IA não foi utilizada neste projeto

O que validei manualmente e como: As validações manuais foram utilizadas para identificar possíveis falhas de código ou de segurança, e, por meio de extensivos tentativas, fui compreendendo e corrigindo para chegar no resultado final.

8. Reflexão Final (5–8 linhas)
O que você melhoraria com mais tempo, principais decisões técnicas e erros que evitaria na próxima. Tudo junto, sem subseções separadas.

Reflexão:

Se eu tivesse mais tempo executaria testes envolvendo mais casos inválidos, assim como reestruturar wait_for_car, de forma a tratar-se de um modelo menos engessado em sua execução (em que basicamente necessita aguardar até o momento em que há a entrada de threads na mutex, autorizando outra thread usar seu mutex, e o recapturando caso/quando despertar). Além disso, também gostaria de ter conseguido inserir condições de verificação mais eficientes tanto em station_load_car quanto em station_on_board. Por fim, não consegui, de forma muito eficiente, inserir verificações para casos de números negativos.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	sim
Rodei os testes e evidencias.log foi gerado	sim
Tenho prints obrigatórios	sim
Testei pelo menos um caso limite e um caso inválido	sim
Preenchi a seção de uso de IA	sim
Revisei o relatório e removi frases genéricas / vazias	sim

10. Se eu tivesse mais 2 horas

Aplique as instruções abaixo de forma objetiva e verificável, evitando texto genérico.

Escreva 5-10 linhas sobre o que você melhoraria (robustez, testes, warnings, performance, organização).

Se eu tivesse mais duas horas, eu melhoraria realizando mais testes do que fiz,
Também utilizaria um modelo arquitetural baseado em flags, o qual era o modelo original de código que eu havia pensado, mas que foi mudado durante o desenvolvimento, logo tornando-se a atual escolha, mais simplificada e menos segura. Também utilizaria mais early-returns e de forma mais eficiente (destacadamente em station_load_car e em station_on_board), o que permitiria obter uma melhor performance. Por fim, também faria melhores verificações para

números negativos e uma massiva reestruturação arquitetural em wait_for_car, de forma a ser menos engessado e mais robusto.

Regras de Integridade (não mudam)

- Ocorrência relevante sem evidência (log ou print) = não conta para a nota.
- Contradição entre prints, log e relato = relatório perde credibilidade.
- Indício de evidência fabricada = entrega invalidada.